

CS543 · WEB SERVICES · TUTORIAL 2 OF 14

Drawing Service Boundaries

One project, end to end — identifying the services inside it, drawing the boundaries, and defining the contracts between them

Module 1 · Tutorial for Lecture 2 · 1 hour · Dr. Pramit Mazumdar · IIIT Vadodara

Recap —Lecture 2

1. Name the five properties of a web service.

Reachable over a network · self-contained · has a contract · independent · loosely coupled.

2. What's the difference between a service and an API?

The API is the menu (what you can call); the service is the kitchen (the whole capability behind it).

3. What two things decide a good service boundary?

Capability (one clear job) and data ownership (it owns its own data — nobody else touches it).

The project: CampusEats

What CampusEats does?

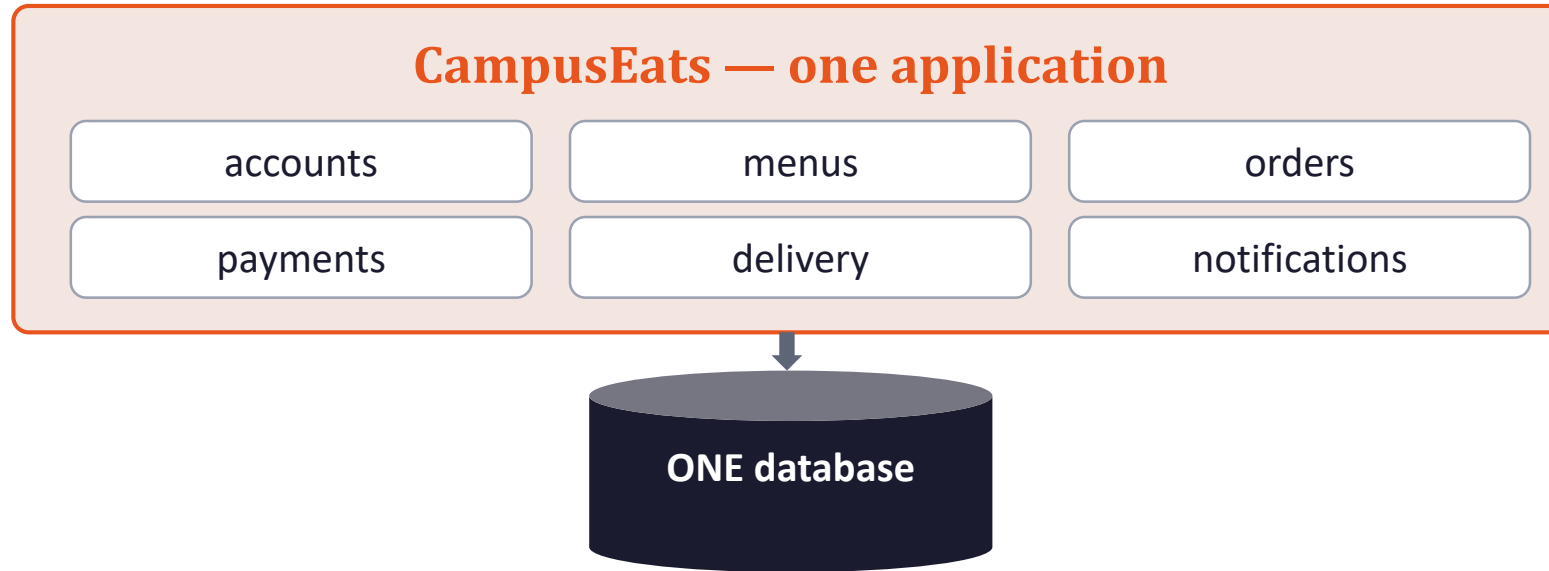
- Students log in and manage a profile and delivery addresses.
- They browse campus restaurants and their menus,
- Add items to a cart, and place an order.
- They pay online.
- A rider is assigned and delivers the food, and the student can track it.
- At each step — order placed, paid, on the way, delivered — the student gets a notification.

The project: CampusEats

What CampusEats does?

- Students **log in and manage a profile** and delivery addresses.
- They browse campus restaurants and their **menus**,
- Add items to a cart, and place an **order**.
- They **pay** online.
- A rider is assigned and **delivers the food**, and the student can track it.
- At each step — order placed, paid, on the way, delivered — the student gets a **notification**.

The starting point: Monolith



How to approach the problem?

1 Find the capabilities

Read the description; underline every distinct job the system does. Each is a candidate capability — a thing the system is responsible for.

2 Group by data ownership

Cluster capabilities that work on the same data. Each cluster becomes one service that exclusively owns that data.

3 Draw the boundaries

One box per service, with a one-line note of what it owns. No two boxes share a table — that's the golden rule.

4 Define the contracts

For each service, state the operations others may call — inputs, outputs, and errors — without exposing anything inside.

Capabilities → data ownership → boundaries → contracts. The same four steps turn any system description into a service design.

Step 1 — underline the capabilities - Tasks

1 manage profile & addresses

2 browse restaurants & menus

3 add to cart & place order

4 pay online / refund

5 assign rider & track delivery

6 send notifications

Step 2 — group by the data each Task uses

Capability	Data it owns	→ becomes
manage profile & addresses	<i>users, addresses</i>	one service
browse restaurants & menus	<i>restaurants, menus, prices</i>	one service
add to cart & place order	<i>carts, orders, status</i>	one service
pay / refund	<i>transactions</i>	one service
assign rider & track	<i>riders, assignments</i>	one service
send notifications	<i>message log</i>	one service

- Each row owns a distinct slice of data with no overlap — so each becomes its own service.
- If data overlaps then we have to merge or re-plan the capabilities.

The services that emerge

Accounts

owns: users, addresses, login

Catalogue

owns: restaurants, menus, prices

Orders

owns: carts, orders, status

Payments

owns: transactions, refunds

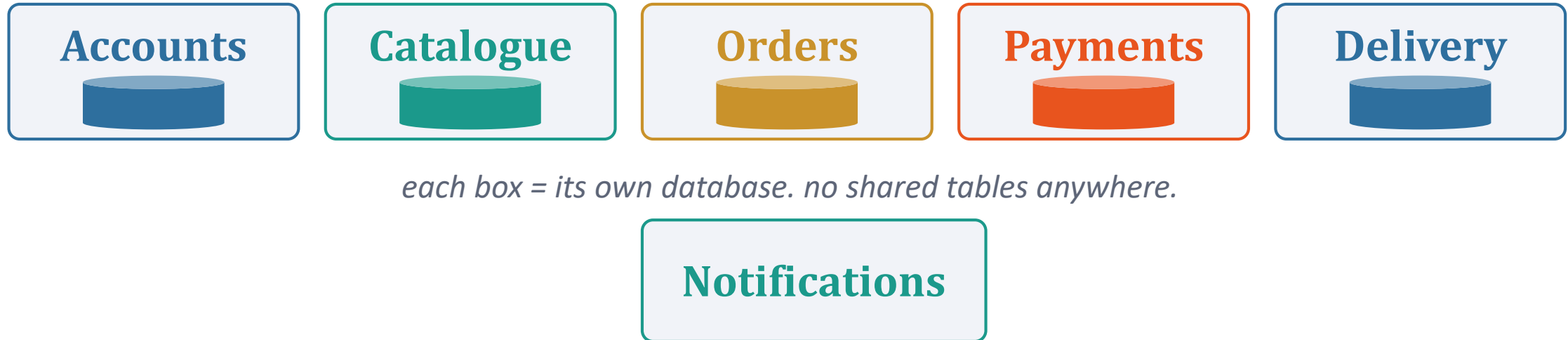
Delivery

owns: riders, assignments

Notifications

owns: message log

The finished boundary diagram



From boundary to contract: what a contract states

- A boundary says what a service owns.
- A contract says what others may ask it to do.
- It's the ONLY thing other services are allowed to depend on — everything inside stays hidden.

The operations

The named things callers can invoke — **placeOrder**, **getMenu**. The verbs the service offers to the outside.

Inputs & outputs

For each operation: what you must send, and what you get back. Precise enough to call without asking.

The errors

What can go wrong and how it's reported — item unavailable, payment declined. Callers must handle these.

What it hides

No table names, no internal ids, no code. Callers depend on the promise, never the implementation.

Contract: the “Orders” service

Operation	Input	Output	Errors
addToCart	studentId, itemId, qty	cart	item unavailable
placeOrder	studentId, cart, addressId	orderId, status, total	empty cart, bad address
getOrder	orderId	order + status	not found
cancelOrder	orderId	status: cancelled	too late to cancel

Notice: no table names, no SQL, no mention of how an order is stored. Just what you can ask, what you send, what you get — the promise, nothing more.

One operation in full: “placeOrder”

IN

- `studentId`
- `items: [{itemId, qty}]`
- `deliveryAddressId`
- `paymentMethodId`

OUT

- `orderId`
- `status: 'placed'`
- `total`
- `estimatedMinutes`

ERRORS

- item unavailable
- payment declined
- invalid address
- empty cart

hidden from callers: the orders table, how ids are generated, which rider gets picked, the payment gateway used

As a group: the Catalogue contract

Operation	Input	Output	Error
listRestaurants	area / filter	restaurants[]	—
getMenu	restaurantId	menu items + prices	not found
checkItem	itemId	available?, price	not found

?

?

?

How the services find and call each other



Why Boundary is important here?

- Orders does NOT read Catalogue's menu table — if it did, the two would be combined together and any change in menu could break orders.
- Instead Orders calls the published checkItem operation and trusts the answer.
- Catalogue can rebuild its entire database overnight - Orders never notices the change.

A request across the services: placing an order



1. Student → Orders.placeOrder
2. Orders → Catalogue.checkItem (still available? price?)
3. Orders → Payments.charge (take the money)
4. Orders → Delivery.assignRider (schedule the drop)
5. Orders → Notifications.send ('order placed') → returns confirmation to student

Orders orchestrates, but owns nothing beyond the order itself. Every fact it needs — price, payment, rider — it gets by calling a contract. That's the design working.

Validate: do the services pass the five properties?

Reachable over a network

✓ Other services call it by its published operations — charge, refund.

Self-contained

✓ It does the whole payment job itself and owns its transactions data.

Has a contract

✓ `charge(amount, method) → receipt; refund(txnId)`. Clear inputs, outputs, errors.

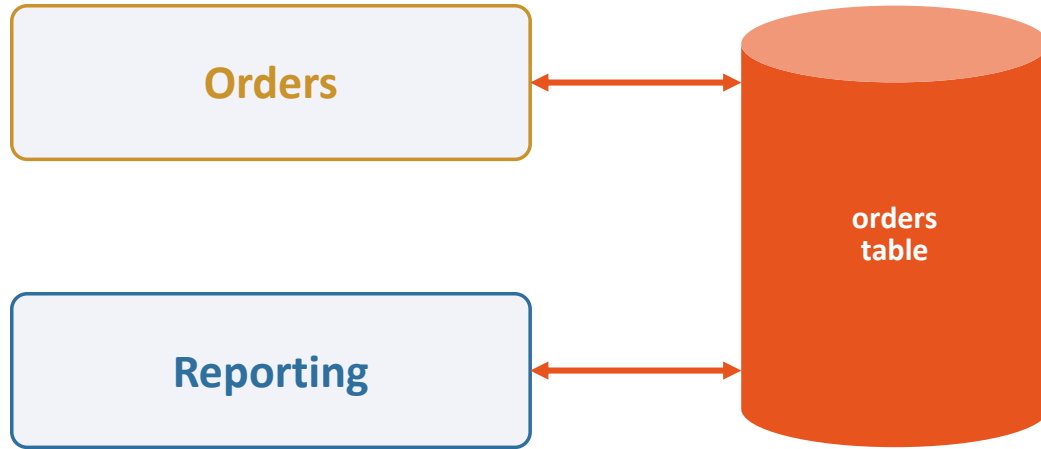
Independent

✓ It can switch payment gateway inside without any caller changing.

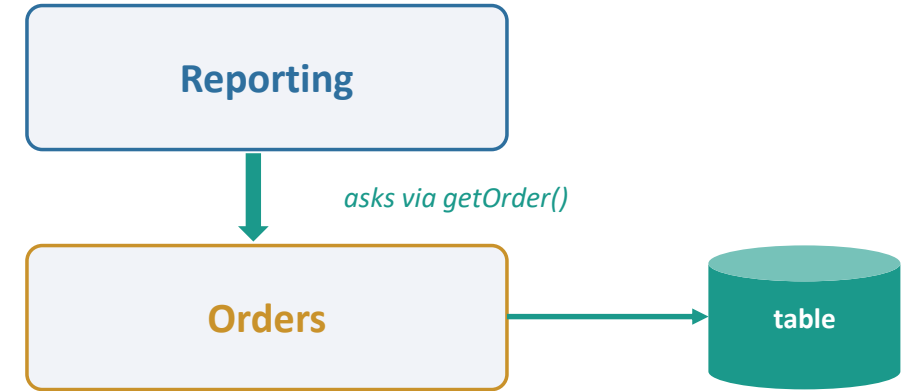
Loosely coupled

✓ Orders never reads its transactions table — only calls charge and trusts the reply.

Spotting a broken boundary — and fixing it



✗ both read AND write it — no single owner



✓ one owner — Orders

- Two services sharing the orders table breaks independence and loose coupling
- Neither can change the schema without breaking the other.
- Solution: Orders owns the table;
- Reporting calls Orders' contract (or reads a copy it owns) instead of touching the table directly.
- One table, one owner — always.

What good looks like: the clinic, worked

Patients

owns: patients, contacts

Appointments

owns: bookings, schedules

Clinical

owns: notes, prescriptions

Billing

owns: invoices, payments

Reminders

owns: message log

`one contract: Appointments`

```
book(patientId, slot)
  → apptId, status
  ! slot taken
cancel(apptId) → ok
```

Conclusion

1

Find the capabilities

underline the jobs — the verbs

2

Group by data ownership

same data → same service; that's the boundary

3

Draw the boundaries

one box per service, one owner per table

4

Define the contracts

operations, inputs, outputs, errors — hide the inside

Capabilities → data ownership → boundaries → contracts. Four steps, any system, every time.